1. The Artifact Export Method (Recommended) Use Colab strictly as your training ground. Train your SARIMA and gradient-boosted regression models on the synthetic data within Colab. Once trained, export the models as .pkl or .joblib files.

- Download these model artifacts from Colab and place them in your VSCode project folder.
- Push the folder to GitHub and deploy it to Streamlit Community Cloud.
- Your Streamlit app will load the pre-trained models directly from the repository to generate predictions instantly, completely severing the need for a live Colab connection.

2. The Data Pipeline Method (The Failsafe)

If the models are too large for GitHub or Streamlit's memory limits, do the forecasting entirely inside Colab.

- Run the models to predict the 24-hour to 7-day patient arrivals.
- Export the forecasted output as a CSV or Parquet file.
- Upload this static forecast file to your Streamlit repository. The Streamlit dashboard will read this file to drive the interactive M/M/c queuing models and the triage optimization engine.

The data generation script is officially complete. You can delete the `TODO NOW: CALIBRATE?` and `TODO NOW: CHECK IF NEED NG RAIN` comments entirely. We successfully anchored the baseline to the 2019 census, and the synthetic rain block provides the exact exogenous variable your SARIMAX model requires.

The remaining `TODO IN PROCESSING` notes in your code represent the architecture for your Google Colab notebook. Now that your .parquet foundation is built, your remaining tasks for the Philippine Junior Data Science Challenge shift entirely to the machine learning and simulation phases.

Here is the exact sequence to build your Colab pipeline:

1. Data Ingestion & Formatting

- Load `synthetic_hospital_inflow.parquet`.
-
- Convert the machine-readable Unix millisecond timestamps back into Pandas Datetime objects (`pd.to_datetime(df['timestamp'], unit='ms')`).
-
- Set the timestamp as the dataframe index so the forecasting algorithm can interpret the hourly frequency.
-

2. The SARIMAX Forecasting Model

- Define `total_arrivals` as your primary target variable.
-
- Define `is_heavy_rain` as your exogenous variable (the "X").
-
- Split the dataset into training and testing blocks, and train the model to predict the next 24 to 48 hours of ER volume.
-

3. The Discrete Event Simulation (DES)

- Build a programmatic queue representing the 70 PGH ER beds and 13 triage nurses.
-
- Feed the SARIMAX-predicted arrivals into this queue hour by hour.
-
- Apply your documented service durations to hold patients in beds, calculating real-time occupancy and wait times.
-

4. The Optimization Engine (Diversion Logic)

- Write a dynamic loop that triggers immediately when the DES hits 70 occupied beds.
-
- Filter the active queue to isolate `acuity_non_urgent` patients.
-
- Cross-reference `alternative_facilities.parquet` and subtract diverted patients from the `max_simultaneous_patients` limits based on the shortest `travel_time_mins`.
-

Are you ready to open Colab and start writing the Python code for Step 1 and Step 2?

Dashboard

- Control the staffing levels, capacity of beds, threshold (patients)

Data Needed

- Facility capacity - The gradient-boosted regression and queuing models need this historical context to mathematically learn how past patient volumes interacted with past capacities to produce those 4-hour wait times
    - Staff (per shift, morning, afternoon, night, etc)
        -
    - Standard no. of beds per hospital's ER - https://www.scribd.com/document/486533925/Key-Regulatory-Issue
        - PGH Director Dr. Gerardo Legaspi previously said that the hospital's emergency room could cater to around 70 patients. - https://newsinfo.inquirer.net/2091416/pgh-overflows-with-patients-directs-public-to-other-hospitals
        -
    - Waiting slots available
- 2-3 years of hourly data on patient arrival and waiting times
    - Which will inherently contain trends to check seasonal arrivals and spikes
        - SARIMA and SARIMAS decompose a single column into:
            - the overall trend (long-term trajectory of patient volume), seasonality (repeating daily or weekly patterns), and residuals (random noise).
        - Arrival seasonality: Program the Poisson distribution to inject volume spikes on **Mondays and Fridays**, which historically register the highest number of emergency department visits. Additionally, configure the daily hourly rates so that the **afternoon shifts** experience the highest patient influx, while weekends (Saturdays and Sundays) see the lowest average arrivals.
- Patient categ: high-acuity, non-critical (percentage)
    - 15% high-acuity (excluded from diversion)
        - 5% resuscitation
        - 10% emergency cases
        - Data pipeline must flag these to exclude from diversion engine
    - 45% urgent
    - 40% non-critical (divertible)
        - 30% less urgent
        - 10% non-urgent cases
        - Primary targets for the constraint optimization model to reroute to the alternative facilities
- Alternative hospitals/clinics for non-critical patients
    - 4 to 5 alternative local health networks or primary care centers

Service Durations (Consultation & Treatment Times): Your queuing models (M/M/c, M/G/c, or discrete-event simulation) require historical service durations alongside arrival rates to estimate queue lengths and server utilization. You should base this on the reported average of roughly 42 minutes of actual consultation time.

Separated Wait Time Metrics: The methodology explicitly states that data needs to be aggregated with separate waiting-time measures for the triage stage and the consultation stage.

Travel Times and Spatial Data: The constrained diversion optimization model minimizes travel times between the main hospital and the alternative facilities. You will need synthetic geographic distances or estimated transit times to your 4-5 alternative centers.

Receiving Facility Constraints: To route patients, the system needs the receiving capacity, service availability, and current congestion levels of the alternative local health networks.

Nurse-to-Patient Ratios: The triage optimization engine requires this data to ensure that diversion recommendations keep staffing within institutional standards.

Exogenous Variables (for SARIMAX): Since you are utilizing SARIMAX (which includes exogenous variables) for your 24-hour to 7-day forecasting, consider generating external synthetic triggers that influence patient volume, such as severe weather events, holidays, or local accident data.

But they do not tell your optimization engine how many patients can actually be treated or how many beds are available at a given time.

For example, an ER with 70 beds and 13 nurses might have 20 occupied beds at one hour and 70 occupied beds at the next. Your dataset will report identical capacity and staffing in both cases.

Your DES pipeline should eventually calculate:

- Patients waiting for triage and consultation.
- Patients receiving treatment.
- Occupied and available beds.
- Patient length of stay, admissions, and departures.
- Resource utilization and time spent above defined saturation thresholds.

You should also distinguish the number of nurses and doctors scheduled from the number actually available for a given patient or task.

1. The SARIMAX Forecasting Model
You will feed `synthetic_hospital_inflow.parquet` into a time-series algorithm to predict the `total_arrivals` for the next 24 hours. You will use `is_heavy_rain` as your exogenous variable (the "X") to train the model to anticipate sudden spikes.

2. The Discrete Event Simulation (DES)
This is where you build a Python `while` loop that simulates the actual physics of the ER queue based on the SARIMAX predictions.

- It will track how many of the 70 beds are occupied.
- It will mathematically "hold" patients in beds for an average of 42 minutes before discharging them.
- It will identify when the ER is mathematically saturated based on the 13 nurses and 3-4 doctors on duty.

3. The Diversion Algorithm (The Optimizer)
When the DES detects that the 70 beds are full, it triggers the routing logic. The algorithm will scan the incoming forecasted patients, isolate the `acuity_non_urgent` cases, and iterate through your `alternative_facilities.parquet` data. It will check the travel times and your baseline capacities to simulate diverting the bottleneck away from PGH.

Are you ready to open a new Colab notebook and start writing the code for the SARIMAX forecasting model, or do you want to map out the DES queueing loop first?

for your competition methodology):

- **Brain 1: The SARIMAX Model (The Machine Learning):** This is the algorithm that actually "learns." It looks at your `synthetic_hospital_inflow.parquet` dataset, studies the historical patterns (Mondays are busy, rain causes spikes), and predicts how many patients will arrive at PGH tomorrow. This model's job is done before the slider is ever touched.
- **Brain 2: The Optimization Engine (The Rule-Based Calculator):** This is where the slider comes in. This engine doesn't learn; it just ruthlessly executes the diversion rules you write based on the math. It takes the patient volume forecasted by Brain 1 and tries to fit them into the available beds.

Python

```python
import streamlit as st

# The slider reads the default "15" from your dataset, but lets the user change it
manila_capacity = st.slider(
    label="Ospital ng Maynila Available Beds",
    min_value=0,
    max_value=30,
    value=15 # This is where your hardcoded baseline comes in handy!
)

# You then pass that variable directly into your queuing simulation
run_diversion_simulation(pgh_forecast, manila_capacity)
```

If this were my PJDSC project...

I'd make only these changes now:

Do now

✅ API error handling

✅ Dynamic expected-hours validation

✅ More conservative rain comments

✅ Simple validation charts/notebook

Defer until after SARIMAX works

⏸️ Negative Binomial

⏸️ Seasonal disease modelling

⏸️ Dynamic staffing

⏸️ Sophisticated flood models

⏸️ Facility capacity calibration

Don't touch

✅ Poisson arrivals

✅ Multinomial acuity allocation

✅ Separate DES stage

✅ Historical weather retrieval

✅ Fixed staffing constraints

✅ Reproducible seed